

LEARNING NUMPY

```
83 on key_press(self, symbol, modifiers):
84     """Delegate key press"""
85     if self.context_index == -1:
86         if symbol == key.UP and not self.active_index == 0:
87             self.menu_labels[self.active_index].color = [255, 255, 255, 255]
88             self.active_index -= 1
89             self.mags_dt = self.get_act_color_mag()
90         elif symbol == key.DOWN and not self.active_index == 1:
91             self.menu_labels[self.active_index].color = [255, 255, 255, 255]
92             self.active_index += 1
93             self.mags_dt = self.get_act_color_mag()
94         else:
95             self.context_index = self.active_index
96     elif symbol == key.ESCAPE:
97         if self.context_index == -1:
98             pygame.app.exit()
99         else:
100             self.context_index = -1
101     elif self.context_index == 1:
102         if symbol == key.ESCAPE:
103             self.context_index = -1
104         else:
105             self.cur_game.on_key_press(symbol, modifiers)
106     else:
107         if symbol == key.ESCAPE:
108             self.context_index = -1
109         else:
110             self.cur_game.on_key_press(symbol, modifiers)
111
```



NumPy Full Course

LEARNING NUMPY PART 1

Introduction to



NumPy is a short for Numerical Python and is a powerful python library for numerical and scientific computing.

It Provides high-performance and offers tools for performing mathematical operations on arrays, including linear algebra and statistical operations.

NumPy is ideal for handling large datasets due to its efficiency compared to Python lists.

Example usage

```
# Installation  
pip install numpy
```

```
import numpy as np  
  
array = np.array([1, 2, 3, 4, 5])  
  
print(type(array))  
  
<class 'numpy.ndarray'>
```



vs



Lists



Structure

- Python lists are flexible and can contain mixed data types.
- NumPy arrays are homogeneous, meaning all elements must be of the same data type.



Performance

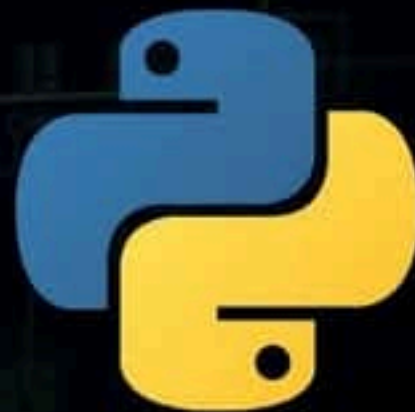
- NumPy arrays are faster due to optimized C operations under the hood.



Memory usage

- NumPy arrays use contiguous memory blocks, making them memory efficient.
- Python lists are arrays of pointers, consuming more memory.

LEARNING NUMPY PART 2



Creating  *NumPy Arrays*

1 Using `np.array()`

One dimension

```
import numpy as np
array = np.array([1, 2, 3, 4])
print(array)
```

```
[1 2 3 4]
```

Multi-dimensional

```
import numpy as np
array = np.array([[1, 2, 3, 4],
                  [5, 6, 7, 8]])
print(array)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

2 Using `np.arange()`

```
import numpy as np

array = np.arange(0,10,2)
print(array)
```

```
[0 2 4 6 8]
```

From 0 to 9 (10-1)
with a step of 2

3

Using `np.linspace()`

```
import numpy as np

array = np.linspace(0,10,5)
print(array)
```

```
[ 0.  2.5  5.  7.5 10.]
```

5 Elements, Lineary
spaced from 0 to 10

4 Using `np.zeros()` and `np.ones()`

```
import numpy as np
```

```
array = np.zeros([2,2])  
print(array)
```

```
[[0. 0.]  
 [0. 0.]
```

```
import numpy as np
```

```
array = np.ones([2,3])  
print(array)
```

```
[[1. 1. 1.]  
 [1. 1. 1.]
```

5 Creating random matrices

```
import numpy as np
```

```
rand_arr = np.random.rand(3, 2)  
print(rand_arr)
```

```
[[0.06142904 0.43901726]  
 [0.5713208  0.23224098]  
 [0.86295519 0.47961164]]
```

```
import numpy as np
```

```
rand_arr = np.random.randn(3, 2)  
print(rand_arr)
```

```
[[ -0.34076564 -1.30347131]  
 [-0.37733825  0.75143626]  
 [ 0.27086002  0.76288207]]
```

```
import numpy as np
```

```
rand_arr = np.random.randint(  
    low=0, high=10, size=(2,3))  
print(rand_arr)
```

```
[[2 7 6]  
 [2 6 0]]
```

A matrix with 3 rows, 2

A matrix with 3 rows, 2

A matrix with 2 rows, 3

columns, following a random

uniform distribution

normal distribution

integers from 0 to 10



Use `np.seed(number)` for
reproductibility when generating
random values

6

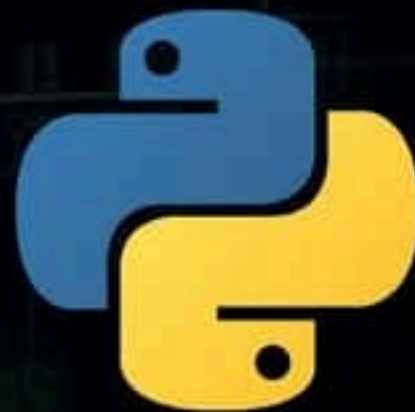
Creating Identity matrices

```
import numpy as np
array = np.eye(3)
print(array)
```

```
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

np.array()	Convert lists/tuples to NumPy arrays.
np.arange()	Generates values with a fixed step.
np.linspace()	Generate Evenly spaced values.
np.zeros() & np.ones()	Create arrays with zeros or ones.
random.rand()	Uniform random values between 0 and 1.
random.randn()	Random integers within a range.
random.randint()	Random integers within a range.
np.eye()	Created identity matrices

LEARNING NUMPY PART 3



NumPy Array Properties and Attributes

1 Getting the shape, size and ndimensions

```
import numpy as np

# 2 Rows, 4 columns
array = np.array([[1,2,3,4],
                  [5,6,7,8]])
print(array.shape)
```

(2, 4)

```
import numpy as np

# 2 Rows * 4 columns = 8
array = np.array([[1,2,3,4],
                  [5,6,7,8]])
print(array.size)
```

8

```
import numpy as np

array1 = np.array([1,2,3,4])
array2 = np.array([[1,2,3,4],
                  [5,6,7,8]])
print(f"array1 dimension : {array1.ndim}")
print(f"array2 dimension : {array2.ndim}")
```

array1 dimension : 1
array2 dimension : 2

← ndim represents the
Number of
← dimensions (axes) in
the array.

2 Flattening Arrays

```
import numpy as np

array = np.array([[1,2,3,4],
                  [5,6,7,8]])
flattened = array.ravel()
print(flattened)
```

[1 2 3 4 5 6 7 8]

← Use np.ravel() to flatten a
← multidimensional array into
a 1D array.

3 Reshaping Arrays

```
import numpy as np
```

```
array = np.array([[1,2,3,4],  
                  [5,6,7,8]])
```

```
reshaped = array.reshape(4,2)  
print(reshaped)
```

```
[[1 2]  
 [3 4]  
 [5 6]  
 [7 8]]
```

Using **np.reshape()**, you can change the shape of an array without altering its data.

```
import numpy as np
```

```
# 2 Rows * 4 columns = 8
```

```
array = np.array([[1,2,3,4],  
                  [5,6,7,8]])
```

```
reshaped = array.reshape(4,-1)  
print(reshaped)
```

```
[[1 2]  
 [3 4]  
 [5 6]  
 [7 8]]
```

In **reshape()**, using -1 automatically infers the dimension based on the array size

4 Checking array data type

```
import numpy as np
```

```
array = np.array([[1,2,3,4],  
                  [5,6,7,8]])
```

```
print(array.dtype)
```

```
int32
```

In this case the output is **int32**, meaning the array contain **integer values** from -2^{31} to $2^{31}-1$. Data type depends on system and input data

5 Converting from one type to another

```
import numpy as np

array = np.array([[1,2,3,4],
                  [5,6,7,8]])
float_array = array.astype(float)
print(float_array)
print(float_array.dtype)
```

```
[[1.  2.  3.  4.]
 [5.  6.  7.  8.]]
float64
```

array.shape	Returns the dimensions of an array.
array.size	Returns the number of elements within an array.
array.ndim	Returns the number of dimensions within of an array
array.reshape()	Reshapes an array to a new shape
array.ravel()	Flatten an array to 1D vector
array.dtype	Returns the data type of array elements.
array.astype()	Changes the data type of array elements.

LEARNING NUMPY PART 4



*Indexing and
Slicing in*  NumPy

1 Slicing in NumPy

```
import numpy as np
array = np.array([[1, 2, 3],
                  [4, 5, 6]])
# first row, second column
print(array[0, 1])
```

1st row, first two
columns

```
print(array[:, 1])
```

```
[2 5]
```

All rows, second
column

```
print(array[0, :2])
```

```
[1 2]
```

2 Filtering Arrays

```
array = np.array([[10, 20, 30],
                  [40, 50, 60]])
```

```
# Apply a filtering
bool_arr = array > 25
```

```
print(bool_arr)
print(array[bool_arr])
```

The output is an
array with the
values that are >
25

```
[30 40 50 60]
```

This filter checks for
values that are higher
than 25

Output :

```
[[False False True]
 [ True  True  True]]
```


3 Advanced Indexing

```
array = np.array([[4,5,6],  
                  [7,8,9]])  
  
# row 0 & col 1 : value 5  
# Row 1 & col 2 : value 9  
fancy = array[[0, 1], [1, 2]]  
print(fancy)
```

```
[5 9]
```

More examples

Example	Result
<code>array[i, j]</code>	Get the element with row i and column j
<code>array[array > value]</code>	Get the elements that are bigger than X
<code>array[condition]</code>	Get the elements that satisfy the condition
<code>array[[i,j], [k,l]]</code>	Get the element (i,j) and the element (k,l)

LEARNING NUMPY PART 5



*Array
Operations in*  NumPy

1 Simple Element wise Operations

```
import numpy as np

array1 = np.array([1, 2, 2])
array2 = np.array([3, 4, 4])

print(array1 + array2) #output is [4 6 6]
print(array1 - array2) #output is [-2 -2 -2]
print(array1 * array2) #output is [3 8 8]
print(array1 / array2) #output is [0.33 0.5 0.5]
```

2 Universal Functions

Numerical functions

Function	Description	Example
<code>np.add(x, y)</code>	Element-wise addition	<code>np.add([1, 2], [1, 2]) → [2, 4]</code>
<code>np.subtract(x, y)</code>	Element-wise subtraction	<code>np.subtract([5, 3], [2, 1]) → [3, 2]</code>
<code>np.multiply(x, y)</code>	Element-wise multiplication	<code>np.multiply([2, 3], [4, 5]) → [8, 15]</code>
<code>np.divide(x, y)</code>	Element-wise division	<code>np.divide([10, 6], [2, 3]) → [5, 2]</code>
<code>np.power(x, y)</code>	Raise elements of x to powers in y	<code>np.power([2, 3], [3, 2]) → [8, 9]</code>